Week 9: Probabilistic inference by tensor contraction

Jin-Guo Liu, Xi-Wei Pan

Abstract: We begin with a reasoning problem: infer disease probabilities from observations in the ASIA model. Tensor diagrams turn its local tables into a calculation; the same language gives an Ising partition function. We then make exact inference cheaper by changing the contraction order. Belief propagation reuses partial calculations; on loops, we judge its approximation against the exact answer.

In class: Feel the problem through ASIA. Then **Imagine** by asking questions; use the tutorial prompts, diagrams and equations to explore the answers with the instructor. **Do** the hands-on work, then inspect and check its results. Allow about 25 minutes for the opening and diagram language, 45 for inference, 35 for contraction cost and 35 for belief propagation, plus a ten-minute break. Begin the builds in class; continue the UAI benchmark application afterward. Tutorial answers and sketches are not exercise deliverables. Only hands-on builds and their verification evidence count; commit after each build.

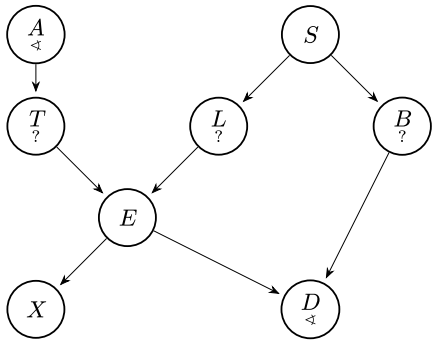
Read further: Tensor-network lecture notes, definitions 2, 3 and 7; Bishop, *Pattern Recognition and Machine Learning*, chapter 8; Roa-Villescas and Liu, *TensorInference*, the ASIA example.

Reasoning from observations

① FEEL

Reasoning uses what we know, together with a model, to infer what we do not know. Here the observations do not determine a unique answer, so we reason with probabilities.

A patient has visited Asia and has difficulty breathing, called *dyspnoea*. Tuberculosis, lung cancer and bronchitis are possible explanations. The ASIA model relates these diseases to smoking, travel and test results. We observe some of these variables and want to infer the others. This is a teaching model, not a diagnostic tool or a model of medical interventions.



Random variable	Meaning
A	Recent trip to Asia
T	Patient has tuberculosis
S	Patient is a smoker
L	Patient has lung cancer
B	Patient has bronchitis
E	Patient has T and/or L
X	Chest X-Ray is positive
D	Patient has dyspnoea

Reproduced from Roa-Villescas and Liu, TensorInference, Figure 1, JOSS 8, 5700, 2023, CC BY 4.0.

The model supplies local probability tables. An arrow points from a parent to a variable whose table depends on it. A, T, S, L denote visit to Asia, tuberculosis, smoking and lung cancer. B, E, X, D denote bronchitis, tuberculosis OR lung cancer, positive X-ray and dyspnoea. Each variable has two *states*: 0 for yes, present or positive, and 1 for no, absent or negative.

- How likely is each disease given the visit and difficulty breathing?
- How does a positive X-ray change those probabilities? If tuberculosis is then confirmed, does lung cancer become more likely, or does tuberculosis already explain the X-ray?
- How can we combine the local tables to answer these questions? What happens when there are too many unknown states to enumerate?

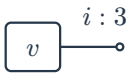
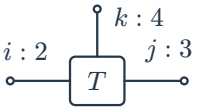
Part 1: Read a tensor network

Goal: translate a diagram into a sum and identify its output. This is an instructor-led tutorial, with no deliverables.

② IMAGINE

The patient's model combines local probability tables. We need a way to show which tables share a variable and which variables to sum. Start with arrays and matrix multiplication, then use the same notation for those tables.

What is a tensor network? What do nodes and edges mean?

Scalar	Vector	Matrix	Rank-3 tensor
			
0 legs	1 leg	2 legs	3 legs
One entry	3 entries	2×3 entries	$2 \times 3 \times 4$ entries

A tensor is an array, drawn as a box. Each leg is an index. The rank counts legs, unlike matrix rank; $i : 2$ means index i has two possible values. Connected indices are summed; open indices remain as output axes.

How does matrix multiplication look in this language?

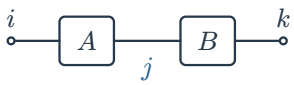
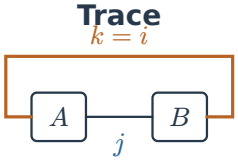
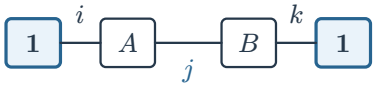
Tensor diagram	Index sum	Einsum
	$C_{ik} = \sum_j A_{ij} B_{jk}$	`ij,jk->ik`
Rank 2 tensors; dimension 2 per leg	Sum j; retain i, k	Output: 2×2 matrix

The shared index j takes the same value in both arrays. Multiply matching entries and sum over j. The free indices i, k remain as output axes.

For $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 2 & 0 \\ 1 & 2 \end{pmatrix}$, the result is $AB = \begin{pmatrix} 4 & 4 \\ 10 & 8 \end{pmatrix}$. Numerical expectations quote the CPU Rust answer key, checked independently with NumPy.

Network notation. The lecture notes write a tensor network as $\mathcal{N} = (\Lambda, \mathcal{T}, V_0)$: all index labels, the input tensors, and the output labels. For this example these are $\{i, j, k\}$, $\{A, B\}$ and $\{i, k\}$. An einsum expression such as `ij,jk->ik` lists the ordered input axes on the left and the output axes on the right. Sum every label omitted from the output.

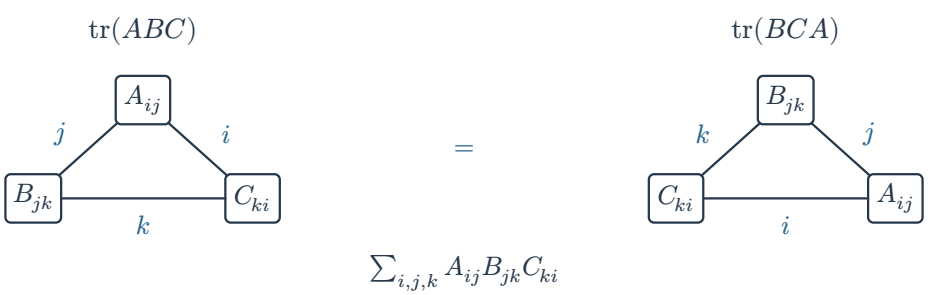
What changes if I join the open legs? What if I sum them independently?

Open output	Trace	Sum of all entries
		
$C_{ik} = \sum_j A_{ij} B_{jk}$	$\sum_{i,j} A_{ij} B_{ji}$	$\sum_{i,j,k} A_{ij} B_{jk}$
Matrix: $\begin{pmatrix} 4 & 4 \\ 10 & 8 \end{pmatrix}$	Scalar: 12	Scalar: 26

Joining the legs identifies $k = i$, giving $\mathrm{tr}(AB) = 12$. Independent sums give 26; each leg connects to its own all-ones vector. Both networks are closed, with no output index, but their sums differ.

Mini-challenge: cyclic traces.

Draw tr(ABC) and tr(BCA). Are they different networks?



$$\sum_{i,j,k} A_{ij} B_{jk} C_{ki}$$

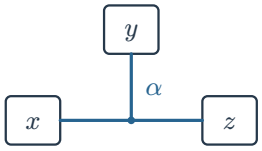
Moving the starting point around the loop preserves every connection and each tensor's row/column labels. Both traces sum the same scalar products. This does not allow arbitrary matrix permutations: $\mathrm{tr}(ACB)$ can differ.

Shared indices. One index may occur in more than two tensors. A hyperedge joins all its occurrences, as in $\sum_\alpha x_\alpha y_\alpha z_\alpha$. All three arrays use the same value of α before it is summed. The junction is one shared index, not another tensor.

Optional: SVD as a tensor diagram. Try this if you know the singular-value decomposition. Otherwise, skip this challenge; inference does not require it.

```
Draw the real SVD M=U diag(s) V^T. Keep s as a vector; hyperedges allowed.
```

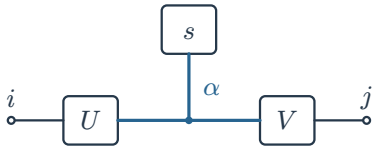
One shared index



$\sum_\alpha x_\alpha y_\alpha z_\alpha$

`a,a,a->`

Real SVD reconstruction



$M_{ij} = \sum_\alpha U_{i\alpha} s_\alpha V_{j\alpha}$

`ia,a,ja->ij`

The SVD supplies orthonormal columns of U, V and nonnegative singular values s. The right-hand diagram reconstructs M: sum α once and leave i, j open. It represents the decomposition, not an algorithm for finding it.

KEY CONCEPTS
- Nodes hold arrays; shared labels require matching index values.
- A network sums the labels not retained in the output.
- Closed networks return scalars. Their connections determine which scalar.

Part 2: Infer disease probabilities in ASIA

Goal: answer each ASIA question with one tensor network of its eight tables, by choosing which indices to fix, sum or keep open; then use the same contraction for an Ising partition function.

Load the ASIA model. From your exercise repository's root, download the data into `week9/` :

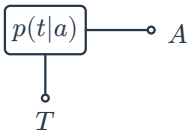
```
mkdir -p week9
cd week9
base=https://raw.githubusercontent.com/TensorBFS/TensorInference.jl
rev=935ed86f8e473f523cac9461c45a1f470e74f0f9
curl --fail --location \
  "$base/$rev/examples/asia-network/model.uai" --output asia.uai
```

② IMAGINE

```
In `asia.uai`, variables 0-7 are A,T,S,L,B,E,X,D, and state 0 means yes. Read the
tables of A (visit to Asia) and T (tuberculosis). How does a confirmed visit change
the tuberculosis probability, compared with unknown travel?
```

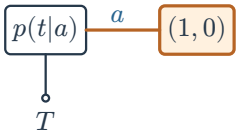
	$A=0$: visit	$A=1$: no visit
$p(A)$	0.01	0.99
$p(T=0\|A)$: TB	0.05	0.01
$p(T=1\|A)$: no TB	0.95	0.99

The table is a tensor



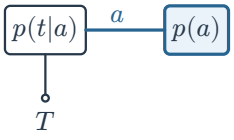
Entry (t, a) is $p(T=t|A=a)$

Visit: fix $A=0$



$p(T=0|A=0) = 0.05$

Unknown travel: sum A

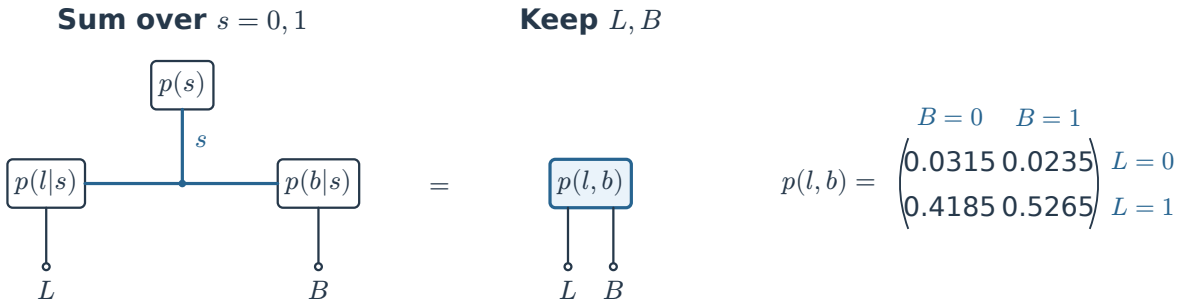


$p(T=0) = 0.0104$

Fixing or summing the travel index. These are the first two tables in `asia.uai`. A confirmed visit attaches $(1, 0)$ to the A leg, which keeps only the $A = 0$ column: $p(T = 0|A = 0) = 0.05$. Unknown travel sums A, weighting the two columns by $p(a)$:

$$p(T = 0) = \sum_a p(a)p(T = 0|a) = \underbrace{0.01}_{p(A=0)} \times 0.05 + \underbrace{0.99}_{p(A=1)} \times 0.01 = 0.0104. \tag{1}$$

> How can a tensor network compute the joint probability of lung cancer and bronchitis when smoking is unknown?

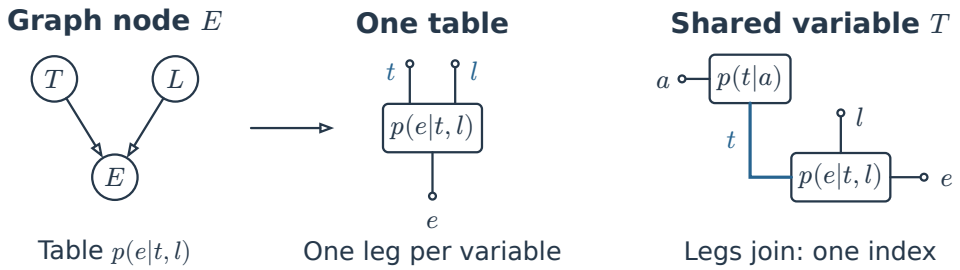


The joint probability of lung cancer and bronchitis. A joint probability describes both diseases together. For each smoking state, multiply the smoking probability by the two disease probabilities, then add the results:

$$p(l, b) = \sum_s p(s)p(l|s)p(b|s). \tag{2}$$

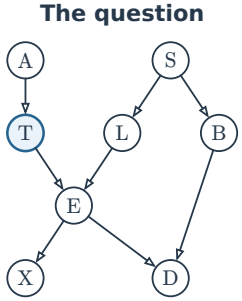
Each table, or *factor*, is a tensor with one leg per variable. All three tables read the same state s, so their s legs join as one shared index. Summing s sums that index; l and b stay open. Evaluating a network this way is called tensor contraction: multiply the entries that share index values, then sum every index that is not open. The einsum `s,ls,bs->lb` writes the same network.

From the graph to the tensor network. Each variable v has one table, the conditional probability of v given its parents, the variables with arrows into v. Its axes are v and those parents: E has parents T, L, so $p(e|t, l)$ has axes e, t, l. The product of the eight tables is the joint probability of all eight variables. Each variable is one index, shared by every table with that axis: T joins $p(t|a)$ and $p(e|t, l)$.



> What sum gives the marginal probability of tuberculosis p(T) from all eight ASIA tables, and how can a tensor network compute it?

The question



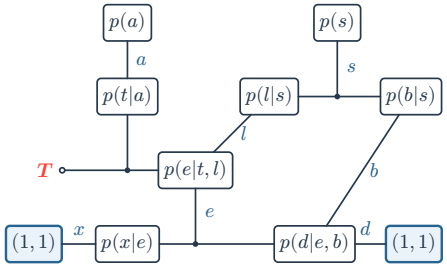
Find T; others unknown

Enumerate: rows with $T = 0$

a	s	l	b	e	x	d	product of 8 entries
0	0	0	0	0	0	0	1.32×10^{-5}
0	0	0	0	0	0	1	1.47×10^{-6}
0	0	0	0	0	1	0	2.70×10^{-7}
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
1	1	1	1	1	1	1	0

Add all 128 rows: $p(T = 0) = 0.0104$

Contract: same sum, no rows



Output: $p(T = t)$

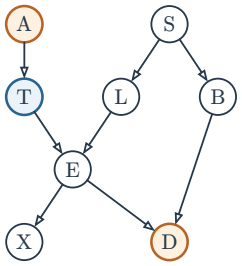
The tuberculosis marginal: assignments and contraction. A marginal probability gives the probability of some variables, with every other variable summed. One row fixes all eight variables; the product of its eight table entries is that row's probability. Adding the rows with $T = 0$ gives $p(T = 0)$:

$$p(t) = \sum_{a,s,l,b,e,x,d} p(a)p(t|a)p(s)p(l|s)p(b|s)p(e|t,l)p(x|e)p(d|e,b). \tag{3}$$

The network joins the eight tables, one shared index per variable, and adds the same products without listing rows. An index used by one table, such as x, is summed by attaching $(1, 1)$. One open index gives a node marginal; several, as for L, B, give a joint marginal. ASIA's $2^8 = 256$ rows can be listed to check the contraction.

> The patient visited Asia and has difficulty breathing. How do these observations change the network for the tuberculosis probability?

The question



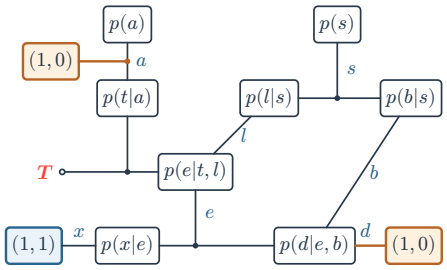
Find T given $A = 0, D = 0$

Enumerate: rows with $T = 0$

a	s	l	b	e	x	d	product of 8 entries
0	0	0	0	0	0	0	1.32×10^{-5}
0	0	0	0	0	1	0	2.70×10^{-7}
0	0	0	0	0	0	1	skip: $d = 1$
1	0	0	0	0	0	0	skip: $a = 1$
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Add the 32 kept rows:
$p(T = 0, A = 0, D = 0) = 0.000395$

Contract: caps fix $A = 0, D = 0$



Output: $p(T = t, A = 0, D = 0)$

Observations select assignments. Attaching $(1, 0)$ to an observed index keeps only its state 0, as the visit did for $p(t|a)$; enumeration skips the rows that disagree. T stays open, giving $p(T = t, \mathcal{E})$, where $\mathcal{E}$ denotes the observations, not the model's variable E. These two values do not sum to one:

$\mathcal{E}$: $A = 0, D = 0$	$p(t, \mathcal{E})$	$p(t\|\mathcal{E})$
Tuberculosis	0.000395	0.087751
No tuberculosis	0.004106375	0.912249

They sum to $p(\mathcal{E}) = 0.004501375$. Dividing by this evidence probability conditions on the observations; the last column is rounded:

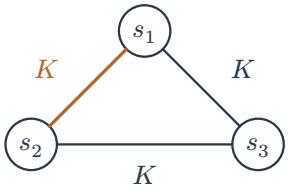
$$p(t|\mathcal{E}) = \frac{p(t, \mathcal{E})}{p(\mathcal{E})}, \quad p(\mathcal{E}) = \sum_t p(t, \mathcal{E}). \tag{4}$$

A closed network supplies the denominator; one open disease index supplies the numerator. A positive X-ray adds a third cap, $(1, 0)$ on X; the hands-on runs compare all three cases.

The Ising partition function. ASIA multiplies local probability tables and sums unknown variables. The Ising model uses the same operation with Boltzmann weights for neighboring spins. Its weights need not sum to one.

> How can the same tensor contraction compute the partition function of a three-spin Ising ring at beta=0.5, J=1, with spins ±1 and no external field?

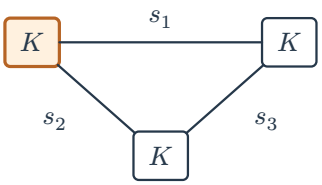
Physical spin ring



Bond → tensor box

Spin → index leg

Tensor network



All spin indices summed: scalar Z

`ij,jk,ki->` with $i = s_1$, $j = s_2$, $k = s_3$

The ring has energy $H = -J(s_1s_2 + s_2s_3 + s_3s_1)$, so each bond supplies the tensor $K(s, t) = \exp(\beta Jst)$. A physical bond becomes a tensor; a spin becomes a shared index.

The partition function sums the product of bond weights over every spin assignment:

$$Z = \sum_{s_1,s_2,s_3} K(s_1,s_2)K(s_2,s_3)K(s_3,s_1) = 2\exp(1.5) + 6\exp(-0.5) = 12.60256209895193. \tag{5}$$

Two configurations have energy -3; the other six have energy 1. Dividing each configuration's weight by Z gives its probability. ASIA's evidence mass and this partition function are both closed tensor contractions. Keeping a spin index open would instead give its two weights.

KEY CONCEPTS

- Every query uses one network: fix observed indices, sum unknown indices, keep query indices open.
- Closing the network gives the evidence probability; dividing by it turns the open joint probabilities into a posterior.
- Joint marginals keep dependence created by a shared cause or a shared effect.
- Ising bond weights use the same contraction as ASIA's probability tables.

③ DO

The model file's table order. The calculation now needs the same tables in code. A factor's *scope* is its ordered list of variables; keep that axis order when loading its array. For example, scope `2 1 0` lists two axes, T, A. Its entries `0.05 0.01 0.95 0.99` fill the tuberculosis table row by row:

File section	Purpose
`MARKOV`	A product of factors; normalization is not implied. These ASIA tables nevertheless form a normalized joint.
First `8`, then dimensions	Eight binary variables.
Second `8`, then scopes	Eight factors; each scope lists its axis count, then ordered IDs.
Table counts and entries	In scope order: each entry count, then its values. Counts are products of axis dimensions; the last axis varies fastest.

Read the left column first, then the right. Column headings are not file contents.

asia.uai

```
1. Header and scopes

MARKOV
8
2 2 2 2 2 2 2 2

8
1 0
2 1 0
1 2
2 3 2
2 4 2
3 5 3 1
2 6 5
3 7 5 4
```

```
2. Probability tables

2  0.01 0.99
4  0.05 0.01 0.95 0.99
2  0.5 0.5
4  0.1 0.01 0.9 0.99
4  0.6 0.3 0.4 0.7
8  1 1 1 0 0 0 0 1
4  0.98 0.05 0.02 0.95
8  0.9 0.7 0.8 0.1 0.1 0.3 0.2 0.9
```

Hands-on: exact inference. Use the downloaded ASIA model to compare no observations (`prior`), a visit and difficulty breathing (`AD`), and those observations plus a positive X-ray (`ADX`). Rust runs the inference; tenferro-rs supplies tensor storage and contraction on the CPU. Use 64-bit floating-point arithmetic to resolve the small numerical differences in the checks. Install the Rust toolchain:

```
rustup toolchain install 1.96.0 --profile minimal
```

```
Prepare a CPU inference project in `week9/` with Rust 1.96.0 and tenferro-rs. Use 64-
bit arithmetic and its tensor, CPU and einsum crates, without GPU or autodiff. Use
Python for plots in `evidence/`; save runs in the requested output directories.
```

Save the following as `infer.design.toml`:

infer.design.toml

```
usage       = "infer --model asia.uai --evidence AD --out artifacts/AD"
description = "infer probabilities from local factors; no unstated defaults"
```

```
[args]
model    = "path to the ASIA UAI model data"
evidence = "prior: no observations; AD: A=0,D=0; ADX: A=0,D=0,X=0"
out      = "directory for this run's evidence"

[writes]
# Keep enumeration and contraction results at full precision for comparison.
"<out>/run.json" = "observations, evidence mass, node marginals and T,L joint"
stdout           = "evidence probability, disease probabilities and discrepancies"
```

```
Build `infer` from `infer.design.toml` to compute ASIA probabilities by enumeration
and tensor contraction. IDs 0-7 are A,T,S,L,B,E,X,D; 0 means yes/present/positive. The
last scope variable changes fastest. Keep tables unchanged. Draw the AD tuberculosis-
query tensor diagram in `evidence/graphs.svg`.
```

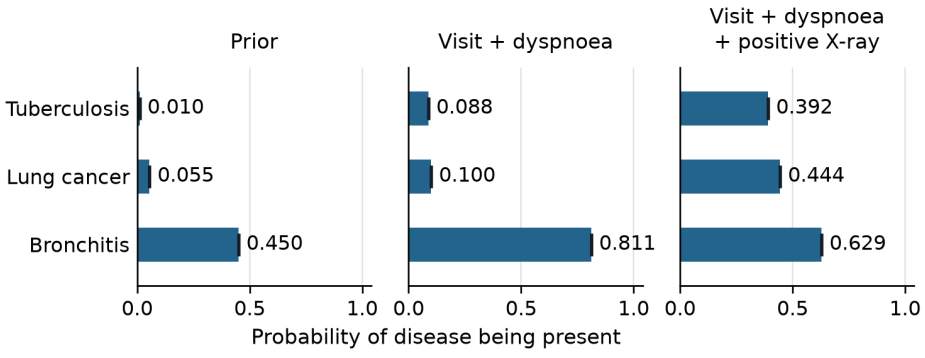
```
Compare tensor contractions with NumPy direct sums for AB and tr(AB), using A=[[1,2],
[3,4]] and B=[[2,0],[1,2]]. Also compare the three-spin Ising-ring partition function
with enumeration: spins ±1, beta=0.5, J=1, no external field. Print both results and
absolute errors; use 1e-12 tolerance.
```

These calculations belong to the inference build, not another project. Read the printed results: $AB = \begin{pmatrix} 4 & 4 \\ 10 & 8 \end{pmatrix}$, $\mathrm{tr}(AB) = 12$ and $Z = 12.60256209895193$. The ring contraction must sum the same eight weights as enumeration, with absolute error below 10^{-12}.

(1) **The diagram retains the intended query.** Open `evidence/graphs.svg`. Each factor must have the axes of its UAI table. Smoking's three occurrences agree; the observations are selected; only T remains open.
(2) **Observations change the disease probabilities.**

```
cargo +1.96.0 install --locked --path .
infer --model asia.uai --evidence prior --out artifacts/prior
infer --model asia.uai --evidence AD --out artifacts/AD
infer --model asia.uai --evidence ADX --out artifacts/ADX
```

```
Plot the prior and posterior disease probabilities, overlaying enumeration and
contraction, in `evidence/posteriors.png`. Compare each T,L joint with the product
of its marginals in `evidence/joint.png`. Print maximum absolute marginal errors
and relative evidence-mass errors against enumeration. Separately label the joint-
minus-product differences as dependence.
```

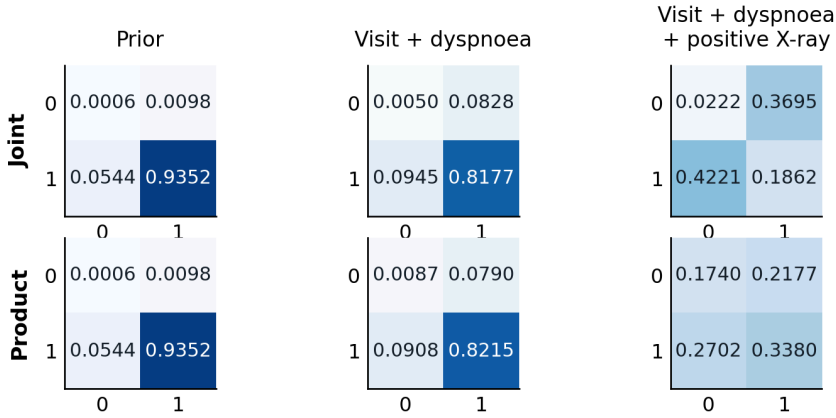


Bars: enumeration. Black marks: tensor contraction.

Adding the positive X-ray raises tuberculosis and lung-cancer probabilities while lowering bronchitis. Evidence masses for prior, AD and ADX are 1, 0.004501375 and 0.00098822675.

Every node and joint marginal sums to one. Require maximum absolute marginal error below 10^{-10} and relative evidence-mass error below 10^{-10}, compared with enumeration at full precision. Also inspect the T, A table: two methods can agree while sharing a misread input.

For ADX, $p(T = 0, L = 0|\mathcal{E}) \approx 0.022214$; the product of the marginals is about 0.174026. Your joint plot must preserve this dependence.



T rows; L columns. 0: present, 1: absent. Rounded to 4 decimals.

Part 3: Make exact contraction cheaper

Goal: explain why the contraction order changes the cost but not the answer, then compare a random and an optimized order for the same ASIA probability.

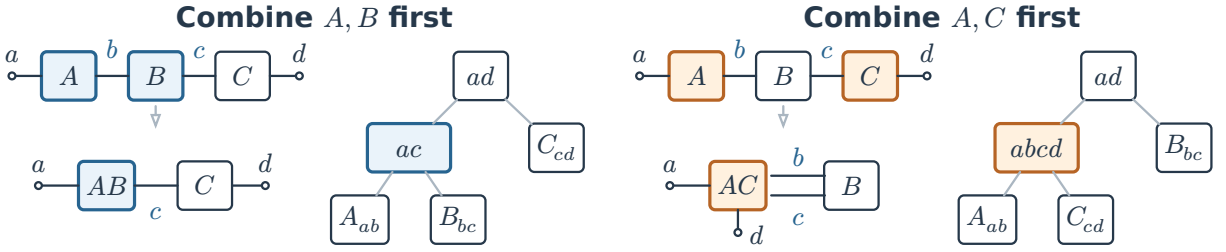
② IMAGINE

Recall from Part 2 that tensor contraction multiplies the entries that share index values, then sums every index that is not open. Enumeration did this for ASIA in one go: 256 rows, each a product of eight entries. Contraction can instead combine the tensors two at a time and sum an index as soon as no other tensor needs it.

A contraction order lists which pair of tensors to combine next. Drawn as a binary tree, the inputs are the leaves and each internal node is one pair step with its result, an *intermediate* tensor.

> For the matrix product ABC, which indices remain after combining A and B? What changes if we combine A and C first?

$$M_{ad} = \sum_{b,c} A_{ab} B_{bc} C_{cd}$$



Colored: the first pair and the intermediate it forms

What an intermediate keeps. A, B share b, and no other tensor uses b, so it is summed: the result keeps a, c. A, C share no index, so nothing is summed, and B still needs b, c: the result keeps a, b, c, d. The rule: an intermediate keeps every index still used outside it or requested in the output, and sums the rest.

> How do we measure the cost of a contraction order? Can we compute it without evaluating the tensors?

Two costs of an order. We measure *time complexity* and *space complexity*. Both depend only on the index sizes, never on the table values, so they can be computed before any contraction. A pair step with index sets I, J, where index i takes d_i values, loops over every combination of their values, one multiply-add each:

$$\text{time complexity} = \sum_{\text{steps}} \prod_{i \in I \cup J} d_i, \quad \text{space complexity} = \max_{\text{tensors}} \text{entries}. \tag{6}$$

For the ABC chain with $n = 10$:

Order	Step	Indices looped over	Time complexity	Result
A, B first	$A \times B$	a, b, c	10^3	ac: 100
	$(AB) \times C$	a, c, d	10^3	ad: 100
	Total		**2000**	space complexity **100**
A, C first	$A \times C$	a, b, c, d	10^4	$abcd$: 10000
	$(AC) \times B$	a, b, c, d	10^4	ad: 100
	Total		**20000**	space complexity **10000**

Time complexity, also called *work*, sets the running time. Space complexity, the size of the largest tensor, decides whether the run fits: 8 bytes per entry, so 30 two-valued indices need 8×2^{30} bytes, 8 GB, about a laptop's memory.

Optional: libraries also report *read–write complexity*, the entries read and written over all steps, which tracks memory traffic.

Choosing an order. We want an order that keeps both small. In Rust, `omeco` picks the order and `tenferro` contracts:

Method	Next pair to combine	Outcome
Random	any pair, chosen at random	baseline; often costly
GreedyMethod	the pair with the smallest result	usually good; not always the best

The saved order is a *plan*: you can redraw it, recount its costs by hand and run it again.

```
Can a good contraction order always prevent large intermediate tensors, or can the
network's structure make them unavoidable?
```

Limits set by the network. First, no order can make a tensor smaller than the largest input: ASIA's $p(e|t, l)$ already has 8 entries, and the greedy plan in the hands-on reaches exactly that. Second, in networks with many loops, every order must at some step keep many indices together, so even the best order can need a large intermediate. This structural limit is measured by the network's *treewidth*. Explore more from the tensor-network lecture notes.

KEY CONCEPTS
- An intermediate keeps an index until no outside tensor or output needs it.
- Judge an order by time complexity and space complexity.
- A better order changes the cost, not the probability.
- The network's structure can limit how small the largest tensor can be.

③ DO

Hands-on: save and analyse contraction orders. Append to `infer.design.toml`:

infer.design.toml
```
[args.ordering]
order   = "scalar evidence order: random or greedy; default greedy"
seed    = "random-order seed; required with random, absent otherwise"
plan    = "replay a saved scalar-query plan; default absent"
analyze = "analyse a saved plan without contraction; default absent"

[writes.ordering]
# Save enough information to replay the plan and independently recount its cost.
"<out>/order.json"      = "contraction plan, planning method and random seed"
"<out>/complexity.json" = """
Work: sum of pair costs, each the product of dimensions in the union of input labels.
Largest tensor: maximum entry count among all inputs and intermediates.
Include per-step costs and retained labels for independent recounting.
"""

# order, plan and analyze are mutually exclusive.
# Analysis writes costs without inference; inference also saves its scalar plan.
```

```
Extend `infer` as `infer.design.toml` specifies: save seeded random and omeco
GreedyMethod orders, replay saved plans, and analyse their contraction complexity
without evaluating tensors.
```

(1) **Random and optimized orders give the same probability at different costs.** Reinstall the extended tool, then compare the same AD query:

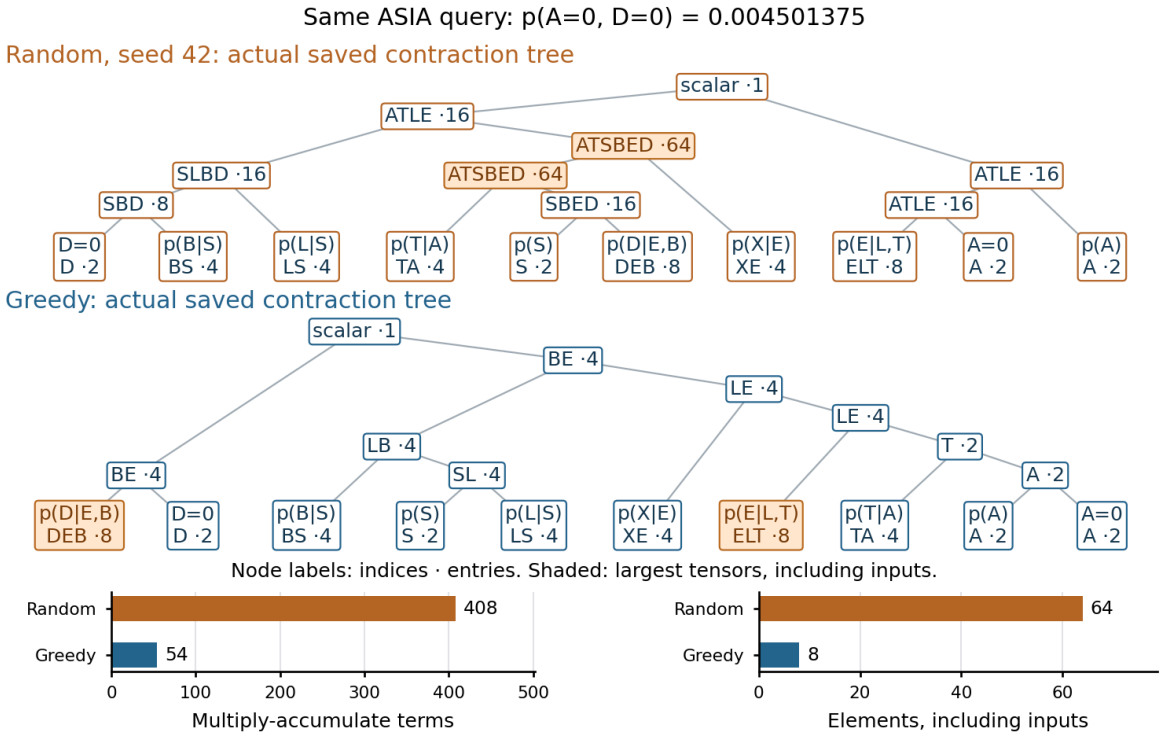
```
cargo +1.96.0 install --locked --path .
infer --model asia.uai --evidence AD --order random --seed 42 --out runs/random
infer --model asia.uai --evidence AD --order greedy --out runs/greedy
infer --model asia.uai --evidence AD \
  --analyze runs/random/order.json --out runs/analysis
infer --model asia.uai --evidence AD \
  --plan runs/greedy/order.json --out runs/replay
```

```
Draw both saved contraction trees and compare work and largest-tensor size in
`evidence/orders.png`. Identify the largest intermediate. Compare replayed
probabilities with enumeration and independently recount the costs.
```

The reference random plan has time complexity 408 and space complexity 64; the greedy plan has 54 and 8: about 7.6 times less work and an 8 times smaller largest tensor. The random plan's first two steps join tables that share no index, like $A \times C$, and hold A, T, S, B, E, D: six two-valued indices.

Both give $p(A = 0, D = 0) = 0.004501375$, within 10^{-10} relative error of enumeration.

Your random generator or tie-breaking may produce different plans. Judge the saved trees and their recounted costs, not a prescribed speedup. An exhaustive search over pair orders finds the smallest possible time complexity: 44, 50 and 52 for prior, AD and ADX. The greedy 54 is close to the best 50 but not optimal.

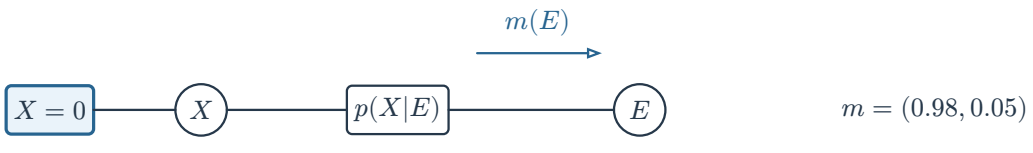


Part 4: Reuse partial contractions with belief propagation

Goal: understand exact messages on a tree, then judge loopy BP by its marginal error as well as its fixed-point residual.

② IMAGINE

Exact tensor contraction can compute marginals, but computing many marginals separately may repeat similar calculations. Can we reuse partial contractions instead of recomputing them?



Sum the branch; retain its connecting index E.

Belief propagation reuses partial contractions as messages. To keep track of who sends each message, draw a *factor graph*: variables are circles and tables are boxes, connected when a table uses a variable. Messages pass along these connections. Combining incoming messages and normalizing estimates a variable's marginal, called its *belief*. A message gives one partial weight for each state of the connecting variable. A positive X-ray sends $(0.98, 0.05)$ toward E; summing an unobserved X-ray instead sends $(1, 1)$. A message need not sum to one.

What advantage does belief propagation offer over exact tensor contraction? What do we give up?

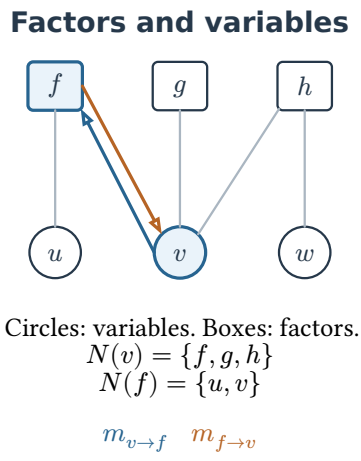
Exact tensor contraction	Loopy belief propagation
Joint tensors on retained indices.	One vector per directed edge; two entries for a binary variable.
Retains correlations; exact answers up to rounding.	Approximate beliefs for all nodes; ignores dependence between incoming branches.
Intermediates can grow exponentially with treewidth.	Each sweep uses local factor tables; messages stay small. Large factors can still be costly.

On a factor tree, BP reuses exact partial contractions. Exact contraction also allows reuse. Loopy BP can save work by assuming independent incoming branches. ASIA's joint-versus-product example already showed why that can change an answer.

Sketch the belief-propagation algorithm.

Initialize messages uniformly. Each *synchronous* sweep reads m and writes $\hat{m}$. Set $m \leftarrow \hat{m}$ and repeat until the maximum message change is below a chosen tolerance or the sweep limit is reached.

Let $N(v)$ be the factors touching variable v, and $N(f)$ the variables in factor f. Write x_A for the states of variables in a set A. Normalize divides a nonnegative vector by its sum; stop if that sum is zero.



Algorithm 1. One sum-product BP sweep

Input: Factor tables f with observations applied; current messages m.
Output: Updated messages $\hat{m}$ and variable beliefs b_v.

for each edge (v, f): variable $\to$ factor **do**
  $\hat{m}_{v\to f}(x_v) \leftarrow \text{Normalize}\left(\prod_{g\in N(v)\setminus\{f\}} m_{g\to v}(x_v)\right)$
end
for each edge (f, v): factor $\to$ variable **do**
  $\hat{m}_{f\to v}(x_v) \leftarrow \text{Normalize}\left(\sum_{x_{N(f)\setminus\{v\}}} f\left(x_{N(f)}\right) \prod_{u\in N(f)\setminus\{v\}} m_{u\to f}(x_u)\right)$
end
for each variable v **do**
  $b_v(x_v) \leftarrow \text{Normalize}\left(\prod_{f\in N(v)} \hat{m}_{f\to v}(x_v)\right)$
end
return $\hat{m}, \{b_v\}$

An empty product is 1. Beliefs use *all* incoming messages; message updates omit the recipient's.

Why is BP exact on a tree?



Orange dashed: removed edge. Circles: variables. Boxes: factors.

Removing an edge separates disjoint branches. Their summaries contain all the needed information without overlap. An inward pass and an outward pass send one message in each direction on every edge. Combining all incoming messages at a variable and normalizing gives its exact marginal.

```
What happens if we run these message updates on ASIA's graph, which has a loop?
```

A message can return through a loop carrying information already counted. The same update equations are then an approximation. A small residual says that the messages have stopped changing under the update, not that their marginals are accurate.

Residual versus error. Let $F(m)$ be the new messages returned by Algorithm 1, the normalized, undamped update. A damped synchronous sweep is $m' = (1-\eta)m + \eta F(m)$. With $\eta = 0.5$, each message averages its old value and the proposed update.

Fixed-point residual	Largest marginal error
$\max\lvert F(m) - m\rvert$	$\max\lvert p_{\text{BP}} - p_{\text{exact}}\rvert$

Evaluate both at the returned state, comparing the same marginals. The residual is not the damped step size $\max|m' - m|$. Damping can stabilize an iteration; it does not make loops exact.

KEY CONCEPTS
- A tree message is an exact partial contraction with one index left open.
- A message omits the recipient's incoming message; a belief uses all incoming messages.
- Convergence and accuracy are different claims on a loopy graph.

③ DO

Hands-on: loopy BP.

Estimate disease probabilities for the same patient cases as exact inference:

Run label	Observations
`prior`	None.
`AD`	Visit to Asia and difficulty breathing.
`ADX`	Visit to Asia, difficulty breathing and a positive X-ray.

Smoking remains unobserved in all three cases. Append this block to the same `infer.design.toml`:

```
infer.design.toml
[args.bp]
method     = "inference method: exact or bp; default exact"
damping    = "fraction of the proposed message used; required with bp only"
tolerance  = "maximum undamped residual for stopping; required with bp only"
max-sweeps = "iteration limit, not a convergence claim; required with bp only"

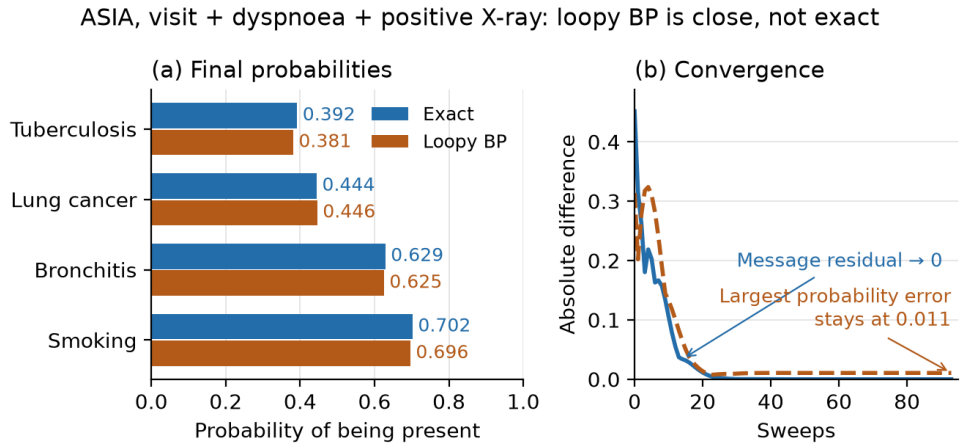
[writes.bp]
# Estimate probabilities for the same model and observations as exact inference.
# Record enough history to compare BP's convergence, accuracy and cost.
"<out>/run.json" = """
Observations, BP settings and final marginals.
At each sweep: node beliefs, undamped residual and cumulative update work.
Normalize every message proposal. Compute the maximum absolute undamped message
change at the same state as the recorded beliefs, including the returned state.
"""
stdout          = "BP sweep, undamped residual and cumulative update work"
```

```
Extend `infer` as `infer.design.toml` specifies to estimate ASIA's disease
probabilities with loopy belief propagation. Apply the observations to the factor
tables. Use uniform initial messages and synchronous updates.
```

(1) **Converged loopy messages can give inaccurate marginals.** Reinstall the extended tool. Run the three cases without overwriting the exact results:

```
cargo +1.96.0 install --locked --path .
infer --model asia.uai --evidence prior --method bp \
  --damping 0.5 --tolerance 1e-10 --max-sweeps 1000 --out artifacts/bp-prior
infer --model asia.uai --evidence AD --method bp \
  --damping 0.5 --tolerance 1e-10 --max-sweeps 1000 --out artifacts/bp-AD
infer --model asia.uai --evidence ADX --method bp \
  --damping 0.5 --tolerance 1e-10 --max-sweeps 1000 --out artifacts/bp-ADX
```

```
Read the saved BP traces and exact-contraction results. Plot message residual and
marginal error against sweeps for each observation case. If the messages stop
changing, are the probabilities necessarily accurate?
```



Residual below 1e-10 after 93 sweeps; the error does not fall with it.

ADX: convergence	ADX: accuracy
Residual 8.23×10^{-11} after 93 sweeps	Marginal error ≈ 0.01061

A nonuniform tree should match enumeration; including the recipient's message fails even there. That is an implementation error, distinct from the approximation error of correctly implemented loopy BP.

Application: A larger inference problem

Goal: test the same reasoning against published answers on UAI 2014 `Promedus_14`. Continue this build after class.

② IMAGINE

This UAI inference benchmark has 414 binary variables, with nine observed. It uses anonymous variable IDs and unnormalized MARKOV potentials, not ASIA's normalized conditional tables. Enumeration would require 2^{414} assignments. A good order can still make exact contraction practical. Which method can give a useful answer at this scale?

```
If we multiply every joint weight by ten, what changes in the partition sum and the
normalized marginals?
```

The sum grows tenfold; normalized marginals stay fixed. Published PR is $\log_{10} Z_{\mathcal{E}}$, so it increases by one. Published MAR contains marginals, so it stays fixed. A lost scale can pass MAR while failing PR.

Weights and normalization. Insert the observed states into every factor that uses them; retain any resulting scalar factors. With unobserved variables x_U,

$$Z_{\mathcal{E}} = \sum_{x_U} \prod_f f\left(x_{N(f)}\right), \quad p(x_q|\mathcal{E}) = \frac{\sum_{x_{U\setminus\{q\}}} \prod_f f\left(x_{N(f)}\right)}{Z_{\mathcal{E}}}. \tag{7}$$

The evidence probability would be $\frac{Z_{\mathcal{E}}}{Z_{\emptyset}}$; the evidence partition sum alone is not a probability. When rescaling intermediate tensors, add each removed multiplier's logarithm to a log scale and restore it in the final PR.

TreeSA and approximate inference. TreeSA searches over contraction trees, sometimes accepting a cost increase to escape a local minimum. Time-only search optimizes arithmetic work. Neither greedy nor TreeSA certifies optimality.

Compare BP's query errors as well as its cost. BP obtains all node beliefs; these exact runs query only three variables, so they do different amounts of work.

KEY CONCEPTS
- PR checks absolute weight scale; MAR checks normalized probabilities.
- Order search can make an exact answer affordable.
- Judge approximate inference by error and cost, not convergence alone.

③ DO

Hands-on: benchmark inference.

In `week9/`, download the model, observations and published answers from the pinned dataset mirror:

```
base=https://raw.githubusercontent.com/mroavi/uai-2014-inference-competition
rev=22005844423f5bca62798548df808cc418fc0d7d
mkdir -p promedus
for suffix in uai uai.evid uai.PR; do
  curl --fail --location \
    "$base/$rev/uai2014/PR/Promedus_14.$suffix" \
    --output "promedus/Promedus_14.$suffix"
done
curl --fail --location \
  "$base/$rev/uai2014/MAR/Promedus_14.uai.MAR" \
  --output promedus/Promedus_14.uai.MAR
```

```
Extend inference to Promedus_14 with its downloaded observations, using omeco greedy
and seeded time-only TreeSA. Save plans and search settings. Compute log10 evidence
partition sum and state-ordered marginals for IDs 59,102,139. Keep MARKOV tables
unchanged and retain log scales.
```

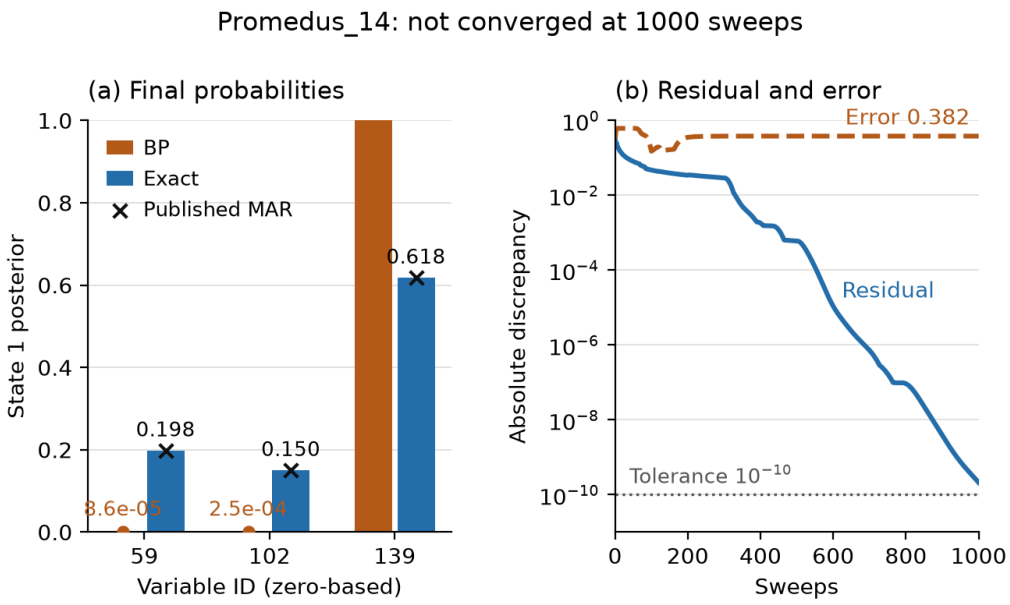
```
Run BP with the ASIA settings. Compare the exact log10 partition sum with PR, and
exact/BP marginals with MAR. Plot order costs and BP residual/error traces. Record
runtime and update work. Show inaccurate and unconverged results.
```

Exact and approximate benchmark answers. Compare exact plans at 10^{-10} absolute tolerance, published PR at 10^{-4} and published MAR at 10^{-5}. The reference has $\log_{10} Z_{\mathcal{E}} = -7.806746304582435$; its state-1 probabilities for IDs 59, 102, 139 are 0.1975882582, 0.1497534323, 0.6175220184.

Reference plan	Work: terms	Largest generated tensor: entries
Greedy	667,111,481	4,194,304
TreeSA	10,635,220	131,072

This TreeSA run reduces work by 62.7 × and tensor size by 32 ×. Recount your own plans; results can differ.

At 1000 sweeps, reference BP still has residual 2.12×10^{-10}, above tolerance, and maximum selected-query error about 0.38235. Its state-1 beliefs are approximately 0.000086, 0.000251, 0.999875. Report it as unconverged and inaccurate for these queries.



Final residual: 2.12e-10 > 1e-10. Sweep limit: 1000.

Hands-on evidence

Show these at the start of Week 10; tutorial answers and sketches are not deliverables:

- ASIA source, query diagram, probability plots and matrix/Ising checks.
- Saved plans and costs; BP residuals and marginal errors.
- `Promedus_14` PR/MAR comparisons, costs and unconverged results.

```
Update README.md with hands-on evidence, reproduction commands, environment, data
sources and settings. Push the implementation and its evidence.
```